

File Allocation Strategies

```
#include <stdio.h>
#include <string.h>

#define BLOCKS 20
#define FILES 5
#define NIL -1

int disk[BLOCKS];
int nxt[BLOCKS]; // for linked allocation

int free_block() {
    for (int i = 0; i < BLOCKS; i++) if (!disk[i]) return i;
    return NIL;
}

void show_disk() {
    printf("Disk: ");
    for (int i = 0; i < BLOCKS; i++) printf("[%d:%s]", i, disk[i]?"X":" ");
    printf("\n");
}

/* — A) SEQUENTIAL — */
struct { char name[20]; int start, len; } sf[FILES];
int sc = 0;

void seq_alloc(const char *name, int n) {
    int cnt = 0, s = NIL;
    for (int i = 0; i < BLOCKS; i++) {
        if (!disk[i]) { if (!cnt) s = i; if (++cnt == n) goto found; }
        else { cnt = 0; s = NIL; }
    }
    printf("No contiguous space\n"); return;
found:
    for (int i = s; i < s+n; i++) disk[i] = 1;
    strcpy(sf[sc].name, name); sf[sc].start = s; sf[sc++].len = n;
    printf("[SEQ] %s: blocks %d-%d\n", name, s, s+n-1);
}
```

```

void seq_show() {
    printf("\n%-15s %-6s %s\n", "File", "Start", "Len");
    for (int i = 0; i < sc; i++)
        printf("%-15s %-6d %d\n", sf[i].name, sf[i].start, sf[i].len);
}

```

/* — B) INDEXED — */

```

struct { char name[20]; int ib, ptr[10], n; } xf[FILES];
int xc = 0;

```

```

void idx_alloc(const char *name, int n) {
    int ib = free_block(); disk[ib] = 1;
    strcpy(xf[xc].name, name); xf[xc].ib = ib; xf[xc].n = n;
    printf("[IDX] %s: idx=%d data=", name, ib);
    for (int i = 0; i < n; i++) {
        int b = free_block(); disk[b] = 1;
        xf[xc].ptr[i] = b; printf("%d ", b);
    }
    xc++; printf("\n");
}

```

```

void idx_show() {
    printf("\n%-15s %-6s %s\n", "File", "Idx", "Data Blocks");
    for (int i = 0; i < xc; i++) {
        printf("%-15s %-6d ", xf[i].name, xf[i].ib);
        for (int j = 0; j < xf[i].n; j++) printf("%d ", xf[i].ptr[j]);
        printf("\n");
    }
}

```

/* — C) LINKED — */

```

struct { char name[20]; int start; } lf[FILES];
int lc = 0;

```

```

void lnk_alloc(const char *name, int n) {
    int head = NIL, prev = NIL;
    for (int i = 0; i < n; i++) {
        int b = free_block(); disk[b] = 1;
        if (prev == NIL) head = b; else nxt[prev] = b;
        nxt[b] = NIL; prev = b;
    }
    strcpy(lf[lc].name, name); lf[lc].start = head;
    printf("[LNK] %s: ", name);
    for (int b = head; b != NIL; b = nxt[b]) printf("%d->", b);
}

```

```
    printf("NULL\n");  
}
```

```
void lnk_show() {  
    printf("\n%-15s %s\n", "File", "Chain");  
    for (int i = 0; i < lc; i++) {  
        printf("%-15s ", lf[i].name);  
        for (int b = lf[i].start; b != NIL; b = nxt[b]) printf("%d->", b);  
        printf("NULL\n");  
    }  
}
```

```
/* — MAIN — */
```

```
int main() {  
    printf("1BM24CS361, Nitya R Swadi\n\n");  
    printf("=== A) SEQUENTIAL ===\n");  
    memset(disk, 0, sizeof(disk));  
    seq_alloc("alpha.txt", 3); seq_alloc("beta.txt", 4); seq_alloc("gamma.txt", 2);  
    seq_show(); show_disk();  
  
    printf("\n=== B) INDEXED ===\n");  
    memset(disk, 0, sizeof(disk));  
    idx_alloc("report.pdf", 4); idx_alloc("notes.txt", 2); idx_alloc("image.png", 5);  
    idx_show(); show_disk();  
  
    printf("\n=== C) LINKED ===\n");  
    memset(disk, 0, sizeof(disk)); memset(nxt, NIL, sizeof(nxt));  
    lnk_alloc("song.mp3", 4); lnk_alloc("video.mp4", 3); lnk_alloc("data.csv", 2);  
    lnk_show(); show_disk();  
}
```

OUTPUT:

```
"C:\Users\Nitya\OneDrive\De X + v
1BM24CS361, Nitya R Swadi

=== A) SEQUENTIAL ===
[SEQ] alpha.txt: blocks 0-2
[SEQ] beta.txt: blocks 3-6
[SEQ] gamma.txt: blocks 7-8

File          Start  Len
alpha.txt     0      3
beta.txt      3      4
gamma.txt     7      2
Disk: [0:X][1:X][2:X][3:X][4:X][5:X][6:X][7:X][8:X][9: ][10: ][11: ][12: ][13: ][14: ][15: ][16: ][17: ][18: ][19: ]

=== B) INDEXED ===
[IDX] report.pdf: idx=0 data=1 2 3 4
[IDX] notes.txt: idx=5 data=6 7
[IDX] image.png: idx=8 data=9 10 11 12 13

File          Idx    Data Blocks
report.pdf    0      1 2 3 4
notes.txt     5      6 7
image.png     8      9 10 11 12 13
Disk: [0:X][1:X][2:X][3:X][4:X][5:X][6:X][7:X][8:X][9:X][10:X][11:X][12:X][13:X][14: ][15: ][16: ][17: ][18: ][19: ]

=== C) LINKED ===
[LNK] song.mp3: 0->1->2->3->NULL
[LNK] video.mp4: 4->5->6->NULL
[LNK] data.csv: 7->8->NULL

File          Chain
song.mp3      0->1->2->3->NULL
video.mp4     4->5->6->NULL
data.csv      7->8->NULL
Disk: [0:X][1:X][2:X][3:X][4:X][5:X][6:X][7:X][8:X][9: ][10: ][11: ][12: ][13: ][14: ][15: ][16: ][17: ][18: ][19: ]
```